LangChain LLM Project Setup Guide

Setting up your Python environment with uv

---

1. What This Guide Covers

You have already seen in the class slides that LangChain is a Python framework which connects a Large Language Model (LLM) with prompts, conversation history, documents and tools. You have also seen the two ways we will call an LLM:

- Local LLM: LangChain → Ollama → Llama 3.2 (runs on your own machine, no internet, no cost)
- Cloud API: LangChain → Groq API → hosted LLM (runs on a server, needs an API key)

This document explains only one thing in detail: how to set up the project environment correctly using uv, and how to run the example code. The Python code itself is shared on GitHub and is already commented, so line-by-line explanation is not repeated here.

2. Before You Start

Install these three things once on your computer.

What	Why we need it	How to check it is installed
Python 3.12	The language our code is written in	python --version
uv	Creates the environment and installs packages	uv --version
Ollama	Runs the LLM locally on your machine	ollama --version
Git	To download code from GitHub	git --version

Installing uv:

Windows (PowerShell):

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
# Standard installation
pip install uv
```

Linux / macOS:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Close the terminal and open it again after installing. Then check with uv --version.

Ollama is downloaded from ollama.com and installed like a normal application.

3. What is uv, and Why Are We Using It

uv is a modern Python package and project manager developed by Astral. It does the same job as pip and venv, but much faster and in one tool.

The important idea is this: every project gets its own separate box of Python packages. This box is called a virtual environment. If one project needs an older version of a library and another project needs a newer version, they do not fight with each other, because each has its own box.

Without a virtual environment, all packages get installed into one place on your computer, and after a few projects it becomes very difficult to fix broken versions. So we always create the environment first, and then install packages inside it.

4. Setting Up the Project Environment

Follow these steps in order. Open a terminal (Command Prompt / PowerShell on Windows, Terminal on Linux or macOS).

Step 1 — Create the project folder

```
mkdir llm-chatbot
cd llm-chatbot
```

Everything from now on happens inside this folder. Do not close the terminal or move to another folder in between.

Step 2 — Create the virtual environment

```
uv venv --python 3.12
```

This creates a hidden folder named .venv inside your project. That folder is the box which will hold all your packages. You never edit anything inside it by hand.

Step 3 — Activate the environment

Windows:

```
.venv\Scripts\activate
```

Linux / macOS:

```
source .venv/bin/activate
```

After activation you will see (llm-chatbot) or (.venv) at the start of your terminal line. That prefix is the confirmation that the environment is active. Verify with:

```
python --version
```

It should print Python 3.12.x. If the prefix is not visible, the environment is not active and packages will get installed in the wrong place.

Step 4 — Create requirements.txt

Create a plain text file named requirements.txt inside the project folder and put these lines in it:

```
langchain-core
langchain-openai
langchain-google-genai
langchain-groq
langchain-ollama
python-dotenv
```

Short meaning of each package:

Package	What it does
langchain-core	The base building blocks of LangChain
langchain-groq	Lets LangChain talk to the Groq cloud API
langchain-ollama	Lets LangChain talk to your local Ollama server
langchain-openai	Connector for OpenAI models (optional for now)
langchain-google-genai	Connector for Google Gemini models (optional for now)
python-dotenv	Reads secret keys from a .env file

Step 5 — Install the packages

```
uv pip install -r requirements.txt
```

Installation takes a short time. If you get an error here, the most common reason is that the environment was not activated in Step 3.

Step 6 — Run a Python file

```
uv run python main.py
```

uv run makes sure the file runs inside the project environment, even if you forgot to activate it.

5. Setting Up the Local Model (Ollama)

The local route needs the model downloaded on your machine once.

```
ollama pull llama3.2
ollama run llama3.2
```

The second command opens a small chat in the terminal. Type something, see the reply, then type /bye to come out. If this works, your local model is ready and LangChain will be able to reach it.

Ollama runs a small server in the background at http://127.0.0.1:11434. Our Python code connects to that address. Keep Ollama running while your code runs.

For the embedding example, also pull the embedding model:

```
ollama pull nomic-embed-text
```

6. Setting Up the Cloud API (Groq)

For the cloud route you need a free API key from console.groq.com. An API key is like a password that tells the service who is calling it.

Never write the key directly inside your Python file. Instead create a file named .env inside the project folder with this single line:

```
GROQ_API_KEY=gsk_your_actual_key_here
```

The code uses load_dotenv() to read this file, so the key stays out of your program and out of GitHub.

Important: add a file named .gitignore in the project folder containing the following, so that your key and your environment folder are never uploaded:

```
.venv/
.env
__pycache__/
```

7. Getting the Example Code from GitHub

The example programs are shared as a GitHub repository. Download them into your project folder:

```
git clone https://github.com/sumansamui/AI-ML-Codes
```

```
cd langchain-tutorial
```

Alternatively, open the repository page in a browser, click the green Code button and choose Download ZIP, then extract it.

Folder layout of the shared code:

File	What it demonstrates
local/LLM_chatbot_local_v1.py	A chatbot using the local Llama 3.2 model through Ollama
local/embedding_model.py	Converting a sentence into a numerical vector
api_call/LLM_reply.py	A single question sent to the Groq cloud model
api_call/LLM_chatbot_v1.py	A chatbot loop using the Groq cloud model

Run any of them from the project folder, for example:

```
uv run python local/LLM_chatbot_local_v1.py
```

Type your question, press Enter, and type exit to stop the chatbot.

Note: these first versions send only the current message to the model. Nothing from the earlier turns is passed. So the chatbot has no memory yet. Memory, prompts, chains and tools come in the next sessions.

Quick Command Summary

```
mkdir llm-chatbot
cd llm-chatbot
uv venv --python 3.12
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Linux / macOS
uv pip install -r requirements.txt
ollama pull llama3.2
uv run python local/LLM_chatbot_local_v1.py
```

Once these steps run without error, your setup is complete and you are ready for the next session on prompts, chains and memory.